

Перешкоди для лами

Лама хоче подорожувати через Андське Плато. У неї є карта плато у вигляді сітки з $N \times M$ квадратних клітинок. Рядки карти пронумеровані від 0 до $N - 1$ зверху вниз, а стовпчики - від 0 до $M - 1$ зліва направо. Клітинка карти в рядку i і стовпчику j ($0 \leq i < N, 0 \leq j < M$) позначається (i, j) .

Лама вивчила клімат плато і виявила, що всі клітинки у кожному рядку карти мають однакову **температуру**, а всі клітинки у кожному стовпчику карти мають однакову **вологість**. Лама передала вам два цілочисельних масиви T та H довжиною N та M відповідно. Тут $T[i]$ ($0 \leq i < N$) вказує на температуру клітинок у рядку i , а $H[j]$ ($0 \leq j < M$) вказує на вологість клітинок у стовпчику j .

Лама також вивчила флору плато і помітила, що клітинка (i, j) є **вільною від рослинності** тоді і тільки тоді, коли її температура перевищує її вологість, формально $T[i] > H[j]$.

Лама може пересуватися по плато, лише слідуючи **допустимими шляхами**. Допустимий шлях - це послідовність окремих клітинок, які задовольняють наступним умовам:

- Кожна пара послідовних клітин на шляху має спільну сторону.
- Всі клітинки на шляху вільні від рослинності.

Ваше завдання полягає у тому, щоб відповісти на Q запитів. Для кожного запиту задано чотири цілих числа: L, R, S , та D . Вам потрібно визначити, чи існує допустимий шлях, такий що:

- Шлях починається у клітинці $(0, S)$ і закінчується у клітинці $(0, D)$.
- Усі клітинки шляху лежать у стовпчиках від L до R включно.

Гарантується, що клітинки $(0, S)$ та $(0, D)$ вільні від рослинності.

Деталі реалізації

Першу функцію, яку ви повинні реалізувати, є наступна:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : масив довжиною N із зазначенням температури у кожному рядку.
- H : масив довжиною M із значеннями вологості у кожному стовпчику.

- Ця функція викликається рівно один раз для кожного тесту, перед будь-якими викликами `can_reach`.

Друга функція, яку ви повинні реалізувати, наступна:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : цілі числа, що описують запити.
- Ця функція викликається Q разів для кожного тесту.

Ця функція повинна повернути `true` тоді і тільки тоді, коли існує допустимий шлях від клітинки $(0, S)$ до клітинки $(0, D)$, такий, що всі клітинки на цьому шляху лежать у стовпчиках від L до R включно.

Обмеження

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ для усіх i , таких, що $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ для усіх j , таких, що $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Обидві клітинки $(0, S)$ та $(0, D)$ є вільними від рослинності.

Підзадачі

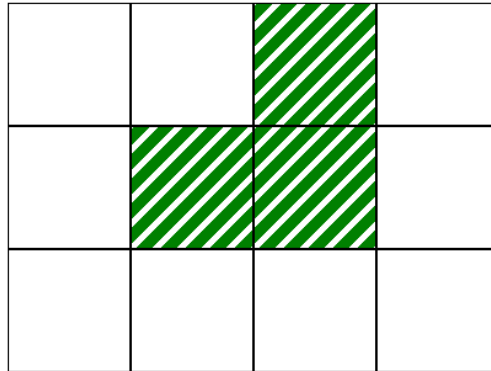
Підзадача	Бали	Додаткові обмеження
1	10	$L = 0, R = M - 1$ для кожного запиту. $N = 1$.
2	14	$L = 0, R = M - 1$ для кожного запиту. $T[i - 1] \leq T[i]$ для усіх i , таких, що $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ для кожного запиту. $N = 3$ та $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ для кожного запиту. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ для кожного запиту.
6	17	Без додаткових обмежень.

Приклад

Розглянемо наступний виклик:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

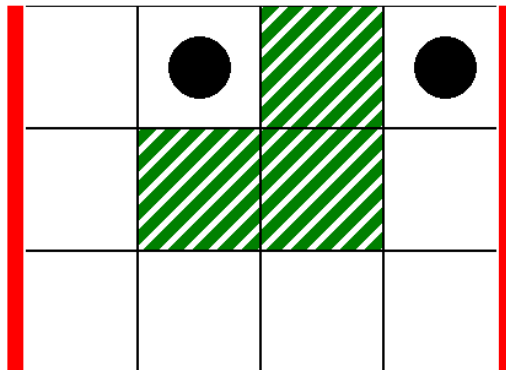
Це відповідає карті на наступному зображенні, де білі клітини вільні від рослинності:



В якості першого запиту розглянемо наступний виклик:

```
can_reach(0, 3, 1, 3)
```

Це відповідає сценарію на наступному зображенні, де товсті вертикальні лінії позначають діапазон стовпчиків від $L = 0$ до $R = 3$, а чорні диски - початкову та кінцеву клітинки:



У цьому випадку лама може дістатися з клітинки $(0, 1)$ до клітинки $(0, 3)$ наступним допустимим шляхом:

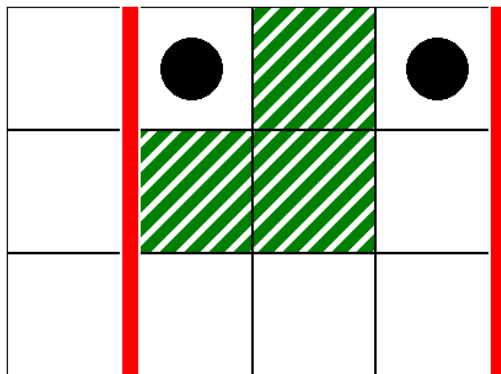
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Отже, цей виклик повинен повернути `true`.

В якості другого запиту розглянемо наступний виклик:

```
can_reach(1, 3, 1, 3)
```

Це відповідає сценарію на наступному зображенні:



У цьому випадку не існує валідного шляху від клітинки $(0, 1)$ до клітинки $(0, 3)$, такого, що всі клітинки цього шляху лежать у стовпчиках від 1 до 3 включно. Тому цей виклик повинен повернути `false`.

Приклад градера

Формат вхідних даних:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Тут $L[k]$, $R[k]$, $S[k]$ та $D[k]$ ($0 \leq k < Q$) задають параметри для кожного виклику для `can_reach`.

Формат вихідних даних:

```
A[0]
A[1]
...
A[Q-1]
```

Тут $A[k]$ ($0 \leq k < Q$) - це 1, якщо виклик `can_reach(L[k], R[k], S[k], D[k])` повернув `true`, і 0 в іншому випадку.